Django-Workshop – Teilnehmerskript 01–06

Praxisprojekt django-advanced-monitoring · Python 3.14, Django 5.2, Django REST Framework und PostgreSQL

- Das Projekt verwaltet Messstationen (`Station`), optionale Stationsmetadaten (`StationMetadata`), Sensoren (`Sensor`) und Messungen (`Measurement`). Stationen haben einen Owner und können mehreren `Tag`-Objekten zugeordnet sein.
- Die Nummern innerhalb jedes Kapitels entsprechen den Abschnitten im Dozentenskript. Die Git-Tags in der Tabelle markieren vorbereitete Code-Stände zum Nachholen. Bei Sphinx ist zusätzlich ein Commit angegeben.

Git und lokale Vorbereitung

```
git clone https://github.com/pietow/django-advanced-monitoring.git
cd django-advanced-monitoring
git fetch origin --tags
git switch -c workshop-01 00-starter
```

- Die Setup-Schritte stehen in der README von `00-starter`. Dort richten Sie die virtuelle Umgebung und `.env` ein, starten PostgreSQL und führen `python manage.py migrate` aus.
- `python manage.py seed_demo_data` legt die Demo-Stationen und Messwerte an. Mit `python manage.py runserver` starten Sie die lokale API, sobald der gewählte Git-Stand API-Routen enthält.

Kapitel	Einstiegs-Tag	Git-Tags im Kapitel / Abschluss-Commit
01 · Serializer	`00-starter`	`01-serializer-basics` → `02-serializer-relations` → `03-serializer-writing` → `04-serializer-complete`
02 · Authentication und Permissions	`04-serializer-complete`	`05-authentication` → `06-permissions`
03 · Migrationen	`07-raw-sql-complete`	`08-migration-basics` → `09-migration-data` → `10-migrations-complete`
04 · Testing	`10-migrations-complete`	`11-testing-bootstrap` → `11-testing-fixtures` → `12-testing-api` → `13-testing-complete` → `14-testing-station-creation`
05 · Packaging	`14-testing-station-creation`	`15-packaging-exercise` → `16-packaging-complete`
06 · Sphinx	`16-packaging-complete`	`17-sphinx-basics` → `a21df91` auf `origin/main`

01 · Serializer

Orientierung im Code: `monitoring/api/serializers.py` und `monitoring/api/views.py`.

1. Serializer und ModelSerializer

- `StationPlainSerializer` deklariert `code`, `name` und `active` explizit und kann auch ein Dictionary darstellen. `StationBasicSerializer` leitet die entsprechenden Felder aus `Station` ab. `Meta.fields` legt fest, welche Felder zum API-Vertrag gehören. Mit `repr(serializer)` sind erzeugte Felder und Validatoren sichtbar.
- Lesen: `StationBasicSerializer(instance=station).data` erzeugt die Ausgabedarstellung. Schreiben: `StationBasicSerializer(data=payload)` nimmt Eingaben entgegen. `initial_data` enthält den ursprünglichen Payload. Nach `is_valid()` stehen die konvertierten Werte in `validated_data` und Fehler in `errors`.
- `.data` ist die Ausgabe. Im Schreibpfad gilt die Reihenfolge `is_valid()` → `save()` → `.data`. Ein Zugriff auf `.data` vor `save()` verhindert den anschließenden Aufruf von `save()`.
- **Git-Tag:** `01-serializer-basics` enthält die ausführbaren Grundlagen.

2. Beziehungen und Repräsentation

Darstellung	Beispiel im Monitoring-Projekt	Entscheidung
`PrimaryKeyRelatedField`	`owner` als Benutzer-ID	Für kompakte, stabile Referenzen. Beim Schreiben begrenzt das `queryset` die auflösbaren IDs.
`StringRelatedField`	Anzeige einer Station über `str(station)`	Für lesbare Ausgaben. Änderungen an `__str__()` wirken sich auf den API-Wert aus.
`SlugRelatedField`	`tags` über `slug_field="name"`	Für fachlich lesbare Referenzen. Bei schreibbaren Slugs ein eindeutiges Feld verwenden.
`HyperlinkedRelatedField`	Sensor mit URL der Detail-View	`view_name` muss zum URL-Namen passen. Der Request im Serializer-`context` ermöglicht absolute URLs.
Nested Serializer	Sensoren als Objekte in der Station	Für Felder, die ein Client zusammen mit der Station benötigt. Große Messreihen eignen sich für einen separaten Endpoint.

- `many=True` kennzeichnet sowohl eine Sammlung am Root, etwa `StationSerializer(stations, many=True)`, als auch eine To-many-Beziehung im Serializer. Eine optionale 1:1-Beziehung wie `metadata` ist dagegen ein einzelnes Objekt.
- `source="metadata.elevation_m"` liest einen Attributpfad. Legen Sie für optionale Metadaten fest, wie ein fehlender Wert dargestellt werden soll. `SerializerMethodField` liefert einen berechneten Read-only-Wert wie `measurement_count`.
- **Git-Tag:** `02-serializer-relations` enthält die Stations- und Sensorrepräsentationen und die zugehörigen API-Views.

3. Validierung und Schreiben

- `StationWriteSerializer.validate_code()` lehnt Codes mit Leerzeichen ab. Objektvalidierung prüft im Kurs die Kombination aus Latitude und Longitude. Beim PATCH können Werte fehlen. `validate()` zieht dann die bestehenden Werte aus `self.instance` heran. Beim POST ist `self.instance` leer.

```
def validate(self, data):
    latitude = data.get("latitude", getattr(self.instance, "latitude", None))
    longitude = data.get("longitude", getattr(self.instance, "longitude", None))
    if latitude == 0 and longitude == 0:
        raise serializers.ValidationError(
            "Latitude and longitude must not both be zero."
        )
    return data
```

- `is_valid(raise_exception=True)` gibt Validierungsfehler im API-Pfad an DRF weiter. `read_only` schließt ein Feld aus der Eingabe aus. `write_only` schließt es aus der Ausgabe aus. `required` bestimmt, ob es bei vollständiger Eingabe erwartet wird. `partial=True` erlaubt das Weglassen von Feldern.
- `save()` ruft mit Instanz `update(instance, validated_data)` auf, andernfalls `create(validated_data)`. Beim einfachen `Serializer` definieren Sie die Persistenz selbst, im Kurs etwa beim `TagWritePlainSerializer`. Der `ModelSerializer` liefert eine Standardimplementierung für gewöhnliche Schreibvorgänge.
- `serializer.save(owner=...)` ergänzt serverseitig gewählte Werte. Im Serializer-Kapitel stammt der Owner aus der Demo-Station. Im nächsten Kapitel verwendet der POST `request.user`.
- Der `StationNestedWriteSerializer` trennt Stationsdaten von `metadata`. Ein Nested Write muss festlegen, wann ein Metadatensatz erstellt wird und wie der PATCH vorhandene Werte behandelt. `transaction.atomic()` umfasst Eltern- und Kindoperationen, damit bei einem Fehler in der Transaktion keine halbfertige Änderung bestehen bleibt.
- **Git-Tag:** `03-serializer-writing` enthält Feld- und Objektvalidierung sowie Nested Writes.

4. Fortgeschrittene Anpassung des Serializer-Verhaltens

- `extra_kwargs` ergänzt Optionen für automatisch erzeugte Felder. Bei explizit deklarierten Feldern stehen die Optionen direkt am Feld. `depth` erzeugt verschachtelte Ausgaben automatisch. Explizite Serializer machen den Umfang der Repräsentation sichtbar.
- `context` trägt Zusatzinformationen. Der `StationRepresentationDemoSerializer` blendet anhand von `include_coordinates` Ausgabefelder aus. `to_representation()` ändert dabei die Ausgabe nach `super().to_representation()`. Bereits geladene Werte und ausgeführte Queries bleiben davon unberührt.
- **Bonus · Query-Performance:** Bei einer Stationsliste einzeln nachgeladene Owner, Metadaten und Sensoren zählen. `select_related()` lädt einzelne Relationen, `prefetch_related()` lädt Sammlungen. Ein `.count()` pro Sensor kann danach noch eigene Abfragen auslösen. Ein annotiertes Aggregat wie `Count` kann die Anzahl mit dem QuerySet laden.
- **Git-Tag:** `04-serializer-complete` enthält die Ausgabedemos und Beispiele zur Query-Optimierung.

5. Entscheidungs-Spickzettel: Welches Serializer-Werkzeug wann?

- IDs oder Slugs eignen sich für Referenzen, Nested Serializer für gemeinsam benötigte Felder. Schreib- und Leseserializer können unterschiedliche Verträge abbilden.
- Eingaben im Serializer validieren, Zugriff an der View beziehungsweise im QuerySet regeln und zusammenhängende Schreiboperationen in einer Transaktion ausführen.

02 · Authentication und Permissions

Orientierung im Code: `config/settings.py`, `monitoring/api/permissions.py` und `monitoring/api/views.py`.

1. Authentication und Authorization

- Authentication identifiziert den Aufrufer und setzt `request.user` sowie je nach Verfahren `request.auth`. Permissions entscheiden anschließend, ob dieser Aufrufer den Request oder die Operation am konkreten Objekt ausführen darf.

2. Authentication in unserem Workshop

- `rest_framework.authtoken` in `INSTALLED_APPS` ergänzen und `python manage.py migrate` ausführen. Im Kurs steht `TokenAuthentication` vor `SessionAuthentication`. Mit `AllowAny` ist die Liste zunächst ohne Token erreichbar. Der spätere Kursstand setzt global `IsAuthenticated`.
- `DEMO-ALPINE-01` gehört im Demo-Szenario `workshop_alice`. `workshop_bob` ist der zweite Benutzer und `workshop_admin` hat `is_staff=True`. In Postman dieselbe Stations-ID und den zur Rolle passenden Token verwenden.

```
python manage.py drf_create_token workshop_alice
python manage.py drf_create_token workshop_bob
python manage.py drf_create_token workshop_admin
```

- Header in Postman: `Authorization: Token <token>`. Tokens gehören in lokale Variablen.

3. Andere Authentication-Verfahren kurz eingeordnet

- `BasicAuthentication` übermittelt Benutzername und Passwort bei jedem Request. `SessionAuthentication` greift auf die Django-Session zurück. Authentifizierte Session-Requests mit zustandsändernden Methoden erfordern einen gültigen CSRF-Nachweis.
- Bei Postman kann ein Session-Cookie einen Test ohne Token beeinflussen. Anonyme Requests mit einem Client ohne Session-Cookie prüfen.

4. Permissions

- `permission_classes` einer View ersetzt die globale Einstellung für diese View. `AllowAny` lässt jeden Request zu. `IsAuthenticated` verlangt eine Identität, `IsAdminUser` prüft `is_staff` und `IsAuthenticatedOrReadOnly` lässt lesende Methoden auch anonym zu.
- **Git-Tag:** `05-authentication` enthält Token/Session und die geschützten Stationsviews vor der Custom Permission.

5. Custom Permissions

- `BasePermission` bietet zwei Prüfstellen. `has_permission()` läuft vor dem Handler. `has_object_permission()` wird für ein geladenes Objekt über `check_object_permissions()` aufgerufen. `SAFE_METHODS` enthält `GET`, `HEAD` und `OPTIONS`.

6. Beispiel: Stationen lesen, nur eigene Station ändern

- Die Kursregel erlaubt angemeldeten Benutzern, alle Stationen zu lesen. Der Owner darf die eigene Station ändern und löschen, Staff jede Station.
- In der eigenen `APIView` kapselt `get_object(request, pk)` das Laden und die Objektprüfung. `GET`, `PATCH` und `DELETE` verwenden diesen gemeinsamen Pfad.

```
def get_object(self, request, pk):
    station = get_object_or_404(Station, pk=pk)
    self.check_object_permissions(request, station)
    return station
```

- **Git-Tag:** `06-permissions` enthält die korrigierte Permission und den gemeinsamen Detailpfad. Die komplette Klasse und ihre Einbindung stehen im Git-Stand.

7. Permissions lösen nicht jedes Zugriffsproblem

- Listen liefern das Ergebnis ihres QuerySets. Falls nur eigene Stationen sichtbar sein sollen, muss das QuerySet entsprechend gefiltert werden.
- Beim POST ist noch kein Objekt vorhanden. Die View entscheidet vor dem Speichern über erlaubte Eingaben und weist den Owner per `serializer.save(owner=request.user)` zu.

8. `401 Unauthorized` und `403 Forbidden`

- Im Workshop ergibt ein anonymer Zugriff auf die geschützte API `401`, weil die zuerst konfigurierte Token-Klasse einen Authentifizierungs-Header liefern kann. Ein authentifizierter Benutzer ohne nötige Berechtigung erhält `403`. Eine nicht vorhandene Station führt im Detailpfad zu `404`.

10. Entscheidungshilfe

Frage	Stelle im Code
Wer ruft auf?	Authentication und `request.user`
Darf der Request zur View?	`has_permission()`
Darf der Aufrufer mit diesem Objekt arbeiten?	`has_object_permission()` über `check_object_permissions()`
Welche Datensätze sind in der Liste sichtbar?	QuerySet
Wer ist Owner eines neuen Objekts?	Serverseitige Zuweisung beim Speichern

03 · Django-Migrationen

Orientierung im Code: `monitoring/models.py` und `monitoring/migrations/`.

1. `makemigrations` und `migrate`

- `makemigrations` vergleicht den aktuellen Model-Code mit dem aus den Migration Files abgeleiteten Project State. Mit `--dry-run --verbosity 2` lässt sich die geplante Datei vorab sehen.
- `migrate` liest den Abhängigkeitsgraphen und die Einträge in `django_migrations`. Es wendet ausstehende Operationen auf die konfigurierte Datenbank an.

2. `showmigrations` – den tatsächlichen Migrationsstand verstehen

- `showmigrations monitoring` zeigt die erkannten Migrationen und markiert angewendete mit `[X]`. `migrate monitoring --plan` zeigt den nächsten Weg vom aktuellen Datenbankstand zum Ziel.

3. `sqlmigrate` – von der Migration zum SQL

- `sqlmigrate` zeigt das von Django für das jeweilige Backend erzeugte SQL. `RunPython` enthält Python-Logik und lässt sich damit nicht als vollständiges SQL vorab anzeigen.

```
python manage.py showmigrations monitoring
python manage.py migrate monitoring --plan
python manage.py sqlmigrate monitoring 0002
```

4. Aufbau eines Migration Files

- `dependencies` beschreibt die Vorgänger einer Migration. `operations` enthält die auszuführenden Schritte und ihre Änderungen am Project State.

5. Migrationen bilden einen Graphen

- Abhängigkeiten bestimmen die Reihenfolge. Parallele Branches können verschiedene Migrationen mit derselben Nummer erzeugen. Entscheidend ist, welche Felder oder Models beim Ausführen vorhanden sein müssen.

6. Schemaänderungen behandeln

- `0002_add_station_description.py` ergänzt `Station.description` als `TextField(blank=True, default="")`. `blank=True` betrifft die Django-Validierung. Der Default liefert auch bei bestehenden Zeilen einen Wert.

7. Migration Files lesen und bewusst prüfen

- Prüfen Sie die `AddField`-Operation, den Default und die Auswirkung auf vorhandene Stationen. Nach dem Anwenden können Sie den Datenbankwert an einer bestehenden Demo-Station lesen.
- **Git-Tag:** `08-migration-basics` enthält `0002` und die Schemaänderung.

8. Migrationen manuell schreiben

- Der Sensorstatus wird in drei Schritten eingeführt. `0003` legt `status` nullable an. `0004` befüllt Bestandsdaten. `0005` macht `status` zum Pflichtfeld.
- `python manage.py makemigrations monitoring --empty --name populate_sensor_status` erzeugt das Gerüst für den mittleren Schritt, wenn `0003` bereits vorhanden ist und `0004` noch fehlt.

9. Data Migrations mit `RunPython`

- `RunPython` führt die Datenoperation aus. Im Kurs erhalten Sensoren abhängig von `active` den Status `active` oder `inactive`. `schema_editor.connection.alias` und `.using(alias)` adressieren die Datenbank, auf der die Migration läuft.

10. Historische Models

- `apps.get_model("monitoring", "Sensor")` lädt das historische Model mit den Feldern am betreffenden Migrationsstand. Der direkte Import aus `monitoring.models` würde den jeweils aktuellen Code laden.

11. Reversible Data Migrations

- Der Rückwärtscode von `0004` setzt den neu eingeführten Status auf `None`. Vorher rollt Django `0005` zurück. Das Feld ist dann wieder nullable.
- **Git-Tag:** `09-migration-data` enthält die Statusmigrationen bis `0005`.

12. Migrationen zurückrollen

- Einen Rückweg zuerst planen. In einer separaten Kursdatenbank kann man zu `0003` zurückgehen und anschließend wieder vorwärts migrieren.

```
python manage.py migrate monitoring 0003 --plan
python manage.py migrate monitoring 0003
python manage.py migrate monitoring
```

13. `RunSQL` – Datenbankoperationen außerhalb normaler Model-Operationen

- `0006_active_station_codes_view.py` legt die PostgreSQL-View `active_station_codes` mit `RunSQL` an. Die View liefert `id` und `code` aktiver Stationen. `reverse_sql` entfernt sie beim Rückweg.

14. Parameter in `RunSQL`

- `RunSQL` kann SQL und gebundene Werte als Tupel erhalten. Das betrifft Datenwerte. Für Datenänderungen mit historischen Models bietet sich `RunPython` an.

15. Django-State und Datenbank-State

- Der Database State beschreibt, was PostgreSQL enthält. Der Project State beschreibt die Model-Struktur im Migrationsgraphen. `state_operations` registriert hier `ActiveStationCode`, während `RunSQL` die View anlegt.

16. Verwaltete und nicht verwaltete Tabellen

- `managed=False` überlässt Django die ORM-Abbildung und lässt die physische Struktur anderweitig verwalten. Eine View kann damit über ein Model gelesen werden.
- **Git-Tag:** `10-migrations-complete` enthält `0006` und das View-Model.

17. Bestehende Datenbanken mit `inspectdb`

- `inspectdb` leitet aus vorhandenen Tabellen Model-Entwürfe ab. Prüfen Sie insbesondere Feldtypen, Primärschlüssel und Relationen.

04 · Testing mit pytest, pytest-django und DRF

Orientierung im Code: `pytest.ini`, `tests/conftest.py`, `tests/unit/`, `tests/integration/` und `tests/api/`.

1. Teststrategie und Test Pyramid

Ebene	Monitoring-Beispiel	Aussage
Unit	Qualitätskennzeichen prüfen	Kleine Regel läuft isoliert ohne Datenbank.
Integration / DB	Station und ihre gespeicherte Zuordnung abfragen	Model, ORM und Constraints arbeiten zusammen.
API / Service	Zugriff auf eine Station je Rolle prüfen	View, Serializer, Permissions und DB greifen zusammen.
E2E / System	Einen vollständigen Ablauf über die Anwendung ausführen	Der reale Nutzungspfad funktioniert über mehrere Komponenten hinweg.

- Wählen Sie die kleinste Testebene, die den gesuchten Fehler erkennen kann. Einen Datenbank-Constraint prüfen Sie beispielsweise durch einen zweiten identischen Stationscode. So sichert der Test eine Projektregel ab.

2. Warum pytest und pytest-django?

- `pytest` sammelt Tests und wertet normale `assert`-Ausdrücke aus. `pytest-django` lädt in diesem Projekt die Settings über `pytest.ini` und verwaltet DB-Zugriff und Testdatenbank.
- `python -m pytest -q` startet die Suite. Einzelne Tests lassen sich über einen Dateipfad oder mit `-k` auswählen.
- **Git-Tag:** `11-testing-bootstrap` enthält `pytest.ini` und die Testabhängigkeiten. Die ersten Testdateien sind ab `11-testing-fixtures` verfügbar.

3. Unit Test: nur Assertion und Verhalten

- Im Unit-Beispiel entscheidet `is_valid_quality_flag()` in `monitoring/testing_examples.py`, ob `valid`, `suspect` oder `invalid` unterstützt wird. Ein gültiger und ein ungültiger Wert prüfen beide Ausgänge.
- Für Codepfade, die bei ungültiger Eingabe eine Exception auslösen sollen, beschreibt `pytest.raises()` die erwartete Exception.

4. Django-Datenbanktests: Integration

- `@pytest.mark.django_db` gibt Datenbankzugriff frei. Eine Fixture kann stattdessen `db` anfordern. pytest-django richtet die Testdatenbank ein und isoliert die normalen DB-Tests voneinander.
- Koordinaten kommen im Projekt aus einem `DecimalField`. Testdaten daher als `Decimal("52.123456")` anlegen. `Measurement.value` ist als `FloatField` für näherungsweise Messwerte modelliert.

5. Fixtures

- `user` legt einen Testbenutzer an und fordert `db` an. `station(user)` baut darauf auf und verwendet diesen Benutzer als Owner. Der Test fordert über seine Parameter nur die benötigten Fixtures an.
- **Git-Tag:** `11-testing-fixtures` enthält die ersten DB-Tests und Basis-Fixtures.

6. DRF API Tests

- Der `APIClient` sendet Requests durch die DRF-Views. Rückgabe und Status lassen sich prüfen. Für den Stationsdetailpfad verwendet der Kurs `reverse("station-detail", args=[station.pk])`.
- `force_authenticate(user=user)` setzt die Testidentität direkt. Um Token-Authentifizierung selbst zu testen, einen echten Token erzeugen und den Header `Authorization: Token ...` mit dem Client senden.
- **Git-Tag:** `12-testing-api` ergänzt die ersten API-Tests.

7. Permission-Matrix und `pytest.mark.parametrize

- `pytest.mark.parametrize` sammelt Kombinationen aus Rolle, HTTP-Methode und erwartetem Status. Die Matrix des Kurses unterscheidet anonymen Zugriff, fremde Station, Owner und Staff.
- Bei langsamen Tests zuerst `python -m pytest --durations=10` einsetzen. Ein schneller Passwort-Hasher gehört ausschließlich in die Testkonfiguration.

8. Abschlussmodell

- In einem API-Test kann selbst eine korrekte Antwort einen falschen Datenbankzustand verdecken. Wenn Persistenz zum Vertrag gehört, nach dem Request zusätzlich relevante DB-Werte prüfen.

9. Teststand zum Abschluss des Basisblocks

- **Git-Tag:** `13-testing-complete` enthält die vollständige Permission-Matrix. `14-testing-station-creation` ergänzt die Tests für Erstellen und Owner-Zuweisung.

05 · Packaging

Orientierung im Code: `pyproject.toml`, `README-package.md` und `requirements-build.txt`.

1. Paketgrenze erklären

- `monitoring` ist der Importname der wiederverwendbaren Django-App. `django-workshop-monitoring` ist der Distributionsname. Der Projektordner `config/`, die lokale `.env` und die Entwicklungsumgebung gehören zum Quellprojekt.
- Die Auslieferung prüfen wir in drei Schritten. Tests im Quellprojekt, Kontrolle des Wheels und Installation in einem unabhängigen Verbraucherprojekt.

2. Metadaten und Paketauswahl

- `[build-system]` wählt setuptools als Build-Backend. `[project]` beschreibt unter anderem Name, Version, README, Python-Version und Laufzeitabhängigkeiten.
- Django und DRF sind Laufzeitabhängigkeiten. PostgreSQL-Unterstützung wird als optionales Extra `postgres` mit psycopg installiert. `build` steht in den Build-Anforderungen.
- `README-package.md` zeigt, wie andere Teams das Paket installieren, in Django registrieren und migrieren. Vor öffentlicher Veröffentlichung muss außerdem die Lizenz geklärt sein.

3. Wheel bauen und untersuchen

- setuptools ermittelt Python-Pakete wie `monitoring.migrations` und `monitoring.management`. Für `monitoring/sql/sensor_health.sql` braucht das Wheel passende Paketdaten. `monitoring/reports.py` lädt diese Ressource beim Import.
- `15-packaging-exercise` ist der Release-Kandidat. `16-packaging-complete` enthält die Paketdaten-Korrektur. Im gespeicherten Tag steht die Versionsnummer weiterhin auf `0.1.0rc1`. Verwenden Sie bei den folgenden Dateipfaden die Version Ihres tatsächlich gebauten Wheels.

```
python -m pip install -r requirements-build.txt
python -m pytest -q
python -m build
Get-ChildItem dist\*.whl
python -m zipfile -l "<Pfad-zum-Wheel>"
```

5. Wheel im Verbraucherprojekt installieren

- In PowerShell einen neuen, leeren Ordner außerhalb des Quellprojekts öffnen. Dort die Verbraucherumgebung und das Django-Projekt anlegen:

```
py -3.14 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install "Django==5.2.17"
django-admin startproject consumer .
python manage.py check
```

- In der aktivierten Verbraucherumgebung das gebaute Wheel samt optionalem PostgreSQL-Treiber installieren:

```
$wheel = "<absoluter Pfad zum gebauten Wheel>"
python -m pip install "${wheel}[postgres]"
python -c "import monitoring; print(monitoring.__file__)"
```

- Der ausgegebene Pfad soll in `site-packages` der Verbraucherumgebung liegen. In `consumer/settings.py` `rest_framework`, `rest_framework.authtoken` und `monitoring.apps.MonitoringConfig` registrieren. Token/Session und `IsAuthenticated` wie in Kapitel 02 konfigurieren. In `consumer/urls.py` `monitoring.api.urls` unter `api/` einbinden.

- Eine separate PostgreSQL-Datenbank wie `monitoring_packaging` für das Verbraucherprojekt konfigurieren. Das Beispiel des Kurses liest dafür `PACKAGING_DB_NAME`, `PACKAGING_DB_USER`, `PACKAGING_DB_PASSWORD`, `PACKAGING_DB_HOST` und `PACKAGING_DB_PORT` aus der Umgebung. Die Django-App- und URL-Einbindung stehen in der Sphinx-Installationsseite. Danach Setup und angewendete Migrationen prüfen:

```
python manage.py check
python -m pip check
python manage.py migrate --plan
python manage.py migrate
python manage.py showmigrations monitoring
```

- Optional die Datenbank mit `python manage.py seed_demo_data` befüllen. Damit lässt sich auch prüfen, ob der Management Command mit dem Wheel ausgeliefert wird.

6. Abschluss und Übergang

- `16-packaging-complete` ist der Einstieg für den Sphinx-Block. Die Anleitung im nächsten Kapitel übernimmt den hier geprüften Ablauf.
- Für eine spätere Verteilung kann dasselbe geprüfte Artefakt über einen Paketindex bereitgestellt werden. Die lokale Installationsprüfung deckt bereits die Paketgrenze und den Verbraucherpfad ab.

06 · Sphinx

Orientierung im Code: `docs/source/conf.py`, `docs/source/index.rst`, `docs/source/installation.rst` und `docs/source/reference.rst`.

0. Vorbereitung

- `16-packaging-complete` liefert den Ausgangscode für den manuellen Aufbau. `requirements-docs.txt` ist ab `17-sphinx-basics` enthalten. Für den Aufbau aus dem Packaging-Tag holen Sie diese Datei aus dem späteren Tag. Verwenden Sie die aktivierte Projektumgebung.

1. Zielgruppe und Dokumentationsarten

- Die Dokumentation richtet sich an Teams, die die Monitoring-App einbinden. Die Installationsseite führt durch das Einrichten. Die Python-Referenz beschreibt einzelne Funktionen und Klassen.

2. Projekt anlegen und erste Seite bauen

- Im Ausgangsstand erzeugt `quickstart` ein Grundgerüst mit getrennten Quellen und Build-Dateien. Wer `17-sphinx-basics` auscheckt, findet dieses Gerüst schon vor.
- Die Startseite steht in `docs/source/index.rst`. `conf.py` setzt den Projektnamen und liest die Version aus `pyproject.toml`.

```
git show 17-sphinx-basics:requirements-docs.txt | Set-Content -Encoding utf8
requirements-docs.txt
python -m pip install -r requirements-docs.txt
python -m sphinx.cmd.quickstart -q -p "Monitoring" -a "Workshop-Team" --sep --no-makefile
--no-batchfile docs
```

3. Installation nachvollziehbar erklären

- `installation.rst` beschreibt Wheel-Installation, benötigte Django-Apps, Authentifizierung, URL-Einbindung, PostgreSQL und Migrationen.
- `index.rst` verlinkt Seite und Referenz über `toctree`. `.. _installation:` definiert ein Ziel, das mit `:ref:` referenziert werden kann.

```
.. toctree::
   :maxdepth: 2

   installation
   reference
```

4. Docstrings als Code-Referenz einbinden

- `autodoc` importiert Python-Module und übernimmt Docstrings. Die Beispiele des Dozentenskripts sind `monitoring.queries.dictfetchall` und `monitoring.api.permissions.IsStationOwnerOrReadOnly`.
- `conf.py` macht den Projektpfad importierbar, richtet Django mit den benötigten Apps ein und ruft `django.setup()` auf. Das folgende Grundgerüst zeigt die dafür entscheidenden Einstellungen.

```
from pathlib import Path
import sys
import tomllib

import django
from django.conf import settings

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT))
project = "Monitoring"
release = tomllib.loads((ROOT /
"pyproject.toml").read_text(encoding="utf-8"))["project"]["version"]
language = "de"
extensions = ["sphinx.ext.autodoc"]
html_theme = "alabaster"

settings.configure(
    INSTALLED_APPS=[
        "django.contrib.auth",
        "django.contrib.contenttypes",
        "monitoring.apps.MonitoringConfig",
    ],
)
django.setup()
```

- Schwere oder fehlerhafte Modulimporte können den Doku-Build beeinträchtigen. Die vollständige Konfiguration steht in `docs/source/conf.py` auf `17-sphinx-basics`.

```
.. autofunction:: monitoring.queries.dictfetchall

.. autoclass:: monitoring.api.permissions.IsStationOwnerOrReadOnly
```

- **Git-Tag:** `17-sphinx-basics` enthält die Installationsseite und diese Python-Referenz.

6. Abschluss

- Der HTML-Build kann auch schon nach dem Anlegen der ersten Seite ausgeführt werden. Er meldet Warnungen und fehlende Querverweise. Danach Navigation, Links und gerenderte Docstrings im Browser prüfen. Der Build führt die Beispiel-Codeblöcke nicht aus.

```
python -m sphinx -b html -W --keep-going -n -E docs/source docs/build/html
Start-Process docs/build/html/index.html
```